
name: eKYC System Architecture

overview: Comprehensive system design for a fullstack eKYC onboarding application with mobile client, backend API, authentication, multi-step onboarding flow, and state management.

todos:

- id: setup-monorepo
content: Initialize monorepo structure with apps/api and apps/mobile directories
status: pending
- id: backend-auth
content: Implement backend authentication system with JWT tokens and refresh flow
status: pending
- id: mobile-auth
content: Create mobile auth store, login screen, and secure token storage
status: pending
- id: onboarding-flow
content: Build 5-step onboarding flow with state persistence and validation
status: pending
- id: theme-system
content: Implement token-based theming with light/dark mode support
status: pending
- id: production-hardening
content: Add refresh-then-retry, validation, logging, and critical tests
status: pending
isProject: false

eKYC Onboarding System - Detailed Architecture & Design

1. High-Level System Architecture

```
graph TB
  subgraph client [Mobile Client - Expo React Native]
    UI[UI Components]
    StateManager[State Management]
    Persistence[Local Persistence]
    APIClient[API Client]
  end

  subgraph backend [Backend API - Node Express]
    Routes[API Routes]
    Middleware[Auth Middleware]
    Validation[Request Validation]
    Business[Business Logic]
    Storage[In-Memory Storage]
    Logger[Structured Logger]
  end

  UI --> StateManager
  StateManager --> Persistence
  StateManager --> APIClient
  APIClient --> HTTP_JSONI_Routes[HTTP/JSONI Routes]
  Routes --> Middleware
  Middleware --> Validation
  Validation --> Business
  Business --> Storage
  Routes --> Logger
```

2. Backend Architecture

2.1 Layered Architecture

```

graph TB
    subgraph presentation [Presentation Layer]
        Router[Express Router]
        ErrorHandler[Global Error Handler]
    end

    subgraph application [Application Layer]
        AuthMiddleware[Auth Middleware]
        ValidationMiddleware[Validation Middleware]
        Controllers[Controllers]
    end

    subgraph domain [Domain Layer]
        AuthService[Auth Service]
        OnboardingService[Onboarding Service]
        VerificationService[Verification Service]
    end

    subgraph infrastructure [Infrastructure Layer]
        UserRepo[User Repository]
        SessionRepo[Session Repository]
        OnboardingRepo[Onboarding Repository]
        LoggerService[Logger Service]
    end

    Router --> AuthMiddleware
    AuthMiddleware --> ValidationMiddleware
    ValidationMiddleware --> Controllers
    Controllers --> AuthService
    Controllers --> OnboardingService
    Controllers --> VerificationService
    AuthService --> UserRepo
    AuthService --> SessionRepo
    OnboardingService --> OnboardingRepo
    VerificationService --> OnboardingRepo
    Controllers --> LoggerService

```

2.2 API Endpoints Structure

- **Auth Endpoints** (/v1/auth)
 - POST /login - User authentication
 - POST /refresh - Token refresh
- **User Endpoints** (/v1)
 - GET /me - Get current user info
- **Onboarding Endpoints** (/v1/onboarding)
 - POST /submit - Submit onboarding data
- **Verification Endpoints** (/v1/verification)

- GET /status - Get verification status

2.3 Error Handling Strategy

Consistent error response format:

```
{
  error: {
    code: string,    // Stable error codes (VALIDATION_ERROR, INVALID_CREDENTIALS, etc.)
    message: string, // Human-readable message
    details?: {       // Optional details
      fieldErrors?: Record<string, string>
    }
  }
}
```

Error codes mapping:

- VALIDATION_ERROR → 400
- INVALID_CREDENTIALS → 401
- TOKEN_EXPIRED → 401
- TOKEN_INVALID → 401
- UNAUTHORIZED → 401
- NOT_FOUND → 404
- INTERNAL_ERROR → 500

2.4 Authentication Flow

```

sequenceDiagram
    participant Client
    participant API
    participant AuthService
    participant SessionRepo

    Client->>API: POST /v1/auth/login
    API->>AuthService: authenticate(email, password)
    AuthService->>SessionRepo: createSession(userId)
    SessionRepo-->>AuthService: session tokens
    AuthService-->>API: user + session
    API-->>Client: 200 OK with tokens

    Note over Client: Store tokens securely

    Client->>API: GET /v1/me (with accessToken)
    API->>AuthService: verifyToken(accessToken)

    alt Token Valid
        AuthService-->>API: userId
        API-->>Client: 200 OK with user data
    else Token Expired
        API-->>Client: 401 TOKEN_EXPIRED
        Client->>API: POST /v1/auth/refresh
        API->>SessionRepo: refreshSession(refreshToken)
        SessionRepo-->>Client: new tokens
        Client->>API: Retry GET /v1/me
    end
end

```

2.5 Data Models & Storage

In-Memory Storage Structure:

```
// Storage interface
interface InMemoryStore {
  users: Map<string, User>;
  sessions: Map<string, Session>;
  onboardingDrafts: Map<string, OnboardingDraft>;
  verificationStatuses: Map<string, VerificationStatus>;
}

// User model
interface User {
  id: string;
  email: string;
  fullName: string;
  passwordHash: string; // Never expose in responses
}

// Session model
interface Session {
  accessToken: string;
  refreshToken: string;
  userId: string;
  expiresAt: string; // ISO 8601
  refreshExpiresAt: string;
}

// Onboarding draft
interface OnboardingDraft {
  profile?: ProfileData;
  document?: DocumentData;
  address?: AddressData;
  consents?: ConsentsData;
}

// Verification status
interface VerificationStatus {
  status: 'NOT_STARTED' | 'IN_PROGRESS' | 'APPROVED' | 'REJECTED' | 'MANUAL_REVIEW';
  updatedAt: string;
  details: {
    reasons: string[];
  };
}
```

3. Mobile Architecture

3.1 State Management Strategy

Using **Zustand** for global state management (recommended for simplicity and TypeScript support):

```

graph TB
  subgraph stores [Global Stores]
    AuthStore[Auth Store]
    ThemeStore[Theme Store]
    OnboardingStore[Onboarding Store]
    VerificationStore[Verification Store]
  end

  subgraph persistence [Persistence Layer]
    SecureStorage[Secure Storage - Tokens]
    AsyncStorage[AsyncStorage - Theme/Draft]
  end

  subgraph components [React Components]
    LoginScreen[Login Screen]
    HomeScreen[Home Screen]
    OnboardingScreens[Onboarding Screens]
    SettingsScreen[Settings Screen]
  end

  AuthStore --> SecureStorage
  ThemeStore --> AsyncStorage
  OnboardingStore --> AsyncStorage

  LoginScreen --> AuthStore
  HomeScreen --> AuthStore
  HomeScreen --> VerificationStore
  OnboardingScreens --> OnboardingStore
  SettingsScreen --> ThemeStore

```

3.2 State Stores Design

Auth Store:

```

interface AuthState {
  status: 'logged_out' | 'logging_in' | 'logged_in' | 'refreshing' | 'expired';
  user: User | null;
  session: Session | null;
  error: string | null;

  // Actions
  login: (email: string, password: string) => Promise<void>;
  logout: () => void;
  refreshSession: () => Promise<boolean>;
  loadPersistedSession: () => Promise<void>;
}

```

Theme Store:

```

interface ThemeState {
  theme: 'light' | 'dark';
  colors: ColorTokens;
  spacing: SpacingTokens;

  // Actions
  toggleTheme: () => void;
  setTheme: (theme: 'light' | 'dark') => void;
}

```

Onboarding Store:

```

interface OnboardingState {
  draft: OnboardingDraft;
  currentStep: number; // 0-4 for 5 steps
  submissionState: 'idle' | 'submitting' | 'success' | 'error';
  error: string | null;

  // Actions
  updateProfile: (data: Partial<ProfileData>) => void;
  updateDocument: (data: Partial<DocumentData>) => void;
  updateAddress: (data: Partial<AddressData>) => void;
  updateConsents: (data: Partial<ConsentsData>) => void;
  setCurrentStep: (step: number) => void;
  submitOnboarding: () => Promise<void>;
  clearDraft: () => void;
  loadPersistedDraft: () => Promise<void>;
}

```

Verification Store:

```

interface VerificationState {
  status: VerificationStatus | null;
  loading: boolean;
  error: string | null;

  // Actions
  fetchStatus: () => Promise<void>;
  clearStatus: () => void;
}

```

3.3 Navigation Structure


```

graph TB
    subgraph unauthenticated [Unauthenticated Stack]
        Login[Login Screen]
    end

    subgraph authenticated [Authenticated Stack - Bottom Tabs]
        Home[Home Screen]
        Settings[Settings Screen]
        Onboarding[Onboarding Flow]
    end

    subgraph onboardingFlow [Onboarding Stack]
        Step1[Profile Step]
        Step2[Document Step]
        Step3[Address Step]
        Step4[Consents Step]
        Step5[Review Submit]
    end

    Login -->|Authenticated| Home
    Home -->|Session Expired| Login
    Home -->|Start Onboarding| Step1
    Step1 --> Step2
    Step2 --> Step3
    Step3 --> Step4
    Step4 --> Step5
    Step5 -->|Submit Success| Home

```

Route Guard Logic:

- Check `authStore.status` before rendering
- If `logged_out` or `expired`, redirect to Login
- Show loading screen during refreshing state
- Display session expired message when redirecting from expired state

3.4 Component Structure

```
apps/mobile/src/
├── components/
│   ├── common/
│   │   ├── Button.tsx
│   │   ├── Input.tsx
│   │   ├── Card.tsx
│   │   └── LoadingSpinner.tsx
│   ├── onboarding/
│   │   ├── ProfileStep.tsx
│   │   ├── DocumentStep.tsx
│   │   ├── AddressStep.tsx
│   │   ├── ConsentsStep.tsx
│   │   └── ReviewStep.tsx
│   └── layout/
│       └── ProtectedRoute.tsx
├── screens/
│   ├── LoginScreen.tsx
│   ├── HomeScreen.tsx
│   ├── OnboardingScreen.tsx
│   └── SettingsScreen.tsx
├── stores/
│   ├── authStore.ts
│   ├── themeStore.ts
│   ├── onboardingStore.ts
│   └── verificationStore.ts
├── services/
│   ├── api/
│   │   ├── client.ts
│   │   ├── auth.ts
│   │   ├── onboarding.ts
│   │   └── verification.ts
│   ├── storage/
│   │   ├── secureStorage.ts
│   │   └── asyncStorage.ts
│   └── interceptors/
│       └── authInterceptor.ts
├── theme/
│   ├── tokens.ts
│   ├── lightTheme.ts
│   └── darkTheme.ts
├── types/
│   └── index.ts
└── navigation/
    └── AppNavigator.tsx
```

4. Critical Data Flows

4.1 Onboarding Submission Flow

```

sequenceDiagram
    participant User
    participant UI
    participant OnboardingStore
    participant APIClient
    participant Backend

    User->>UI: Fill Profile Step
    UI->>OnboardingStore: updateProfile(data)
    OnboardingStore->>OnboardingStore: Persist to AsyncStorage

    User->>UI: Navigate Next
    UI->>OnboardingStore: setCurrentStep(1)

    Note over User,UI: Repeat for all steps

    User->>UI: Click Submit on Review
    UI->>OnboardingStore: submitOnboarding()
    OnboardingStore->>OnboardingStore: Set state: submitting
    OnboardingStore->>APIClient: POST /v1/onboarding/submit
    APIClient->>Backend: Request with Bearer token

    alt Token Valid & Data Valid
        Backend-->>APIClient: 200 OK
        APIClient-->>OnboardingStore: Success
        OnboardingStore->>OnboardingStore: Clear draft
        OnboardingStore-->>UI: Success
        UI-->>User: Navigate to Home
    else Token Expired
        Backend-->>APIClient: 401 TOKEN_EXPIRED
        APIClient->>APIClient: Attempt refresh
        APIClient->>Backend: POST /v1/auth/refresh
        Backend-->>APIClient: New tokens
        APIClient->>Backend: Retry submit
        Backend-->>APIClient: 200 OK
        APIClient-->>OnboardingStore: Success
    else Validation Error
        Backend-->>APIClient: 400 VALIDATION_ERROR
        APIClient-->>OnboardingStore: Error with field details
        OnboardingStore-->>UI: Display errors
        UI-->>User: Show field-level errors
    end
end

```

4.2 Token Refresh Strategy

Single Retry with Refresh:

```

// Axios interceptor pattern
apiClient.interceptors.response.use(
  (response) => response,
  async (error) => {
    const originalRequest = error.config;

    // Check if 401 and not already retried
    if (error.response?.status === 401 && !originalRequest._retry) {
      originalRequest._retry = true;

      try {
        // Attempt refresh
        const newSession = await authStore.refreshSession();

        if (newSession) {
          // Update token and retry
          originalRequest.headers.Authorization = `Bearer ${newSession.accessToken}`;
          return apiClient(originalRequest);
        }
      } catch (refreshError) {
        // Refresh failed - logout user
        authStore.logout();
        throw refreshError;
      }
    }

    return Promise.reject(error);
  }
);

```

4.3 State Synchronization (Global ↔ Local)

Pattern for Form Components:

```
// Example: ProfileStep.tsx
const ProfileStep = () => {
  const { draft, updateProfile } = useOnboardingStore();

  // Local state for immediate UI updates
  const [localData, setLocalData] = useState(draft.profile || {});

  // Sync local to global on blur or step change
  const handleBlur = () => {
    updateProfile(localData);
  };

  // Sync global to local on mount
  useEffect(() => {
    setLocalData(draft.profile || {});
  }, [draft.profile]);

  return (
    <View>
      <Input
        value={localData.fullName}
        onChange={({value}) => setLocalData({...localData, fullName: value})}
        onBlur={handleBlur}
      />
    </View>
  );
};
```

5. Security Considerations

5.1 Token Management

- **Access Token:** Short-lived (15 minutes), stored in memory or secure storage
- **Refresh Token:** Longer-lived (7 days), stored in secure storage only
- Never log tokens or expose in error messages
- Clear tokens on logout
- Validate token expiry before requests

5.2 Backend Security

- Validate all inputs server-side
- Never trust client data
- Use parameterized queries if using DB (not applicable for in-memory)
- Implement rate limiting for auth endpoints
- Don't leak internal errors (stack traces) to client
- Sanitize PII from logs

5.3 Mobile Security

- Use Expo SecureStore for sensitive data (tokens)
- Use AsyncStorage for non-sensitive data (theme, draft)

- Clear sensitive state on logout
- Implement biometric auth (bonus)
- Certificate pinning for production (bonus)

6. Testing Strategy

6.1 Backend Tests

Unit Tests:

- Services: Auth logic, validation logic
- Middleware: Token verification, error handling
- Repositories: CRUD operations

Integration Tests:

- Auth flow: login → access protected route
- Submit validation: field errors returned correctly
- Token refresh: expired token → refresh → retry
- Error format consistency

Key Test Cases:

```
// test_login_success_and_me_success
describe('Auth Flow', () => {
  it('should login and access protected route', async () => {
    const loginRes = await request(app)
      .post('/v1/auth/login')
      .send({ email: 'test@test.com', password: 'pass123' });

    expect(loginRes.status).toBe(200);
    const { accessToken } = loginRes.body.session;

    const meRes = await request(app)
      .get('/v1/me')
      .set('Authorization', `Bearer ${accessToken}`);

    expect(meRes.status).toBe(200);
    expect(meRes.body.email).toBe('test@test.com');
  });
});

// test_submit_validation_errors
describe('Onboarding Submit', () => {
  it('should return field errors for invalid data', async () => {
    const res = await request(app)
      .post('/v1/onboarding/submit')
      .set('Authorization', `Bearer ${validToken}`)
      .send({ draft: { profile: {} } }); // Missing required fields

    expect(res.status).toBe(400);
    expect(res.body.error.code).toBe('VALIDATION_ERROR');
    expect(res.body.error.details.fieldErrors).toBeDefined();
  });
});
```

6.2 Mobile Tests

Unit Tests:

- Store actions: state transitions
- Utilities: form validation, date formatting
- API client: request formatting, error parsing

Integration Tests:

- Auth store: login → logout → refresh flow
- Onboarding store: draft updates → persistence → submission
- Navigation: route guards work correctly

Key Test Cases:

```

// test_auth_refresh_then_retry
describe('Auth Store', () => {
  it('should refresh token and retry on 401', async () => {
    const store = useAuthStore.getState();

    // Mock expired token response
    mockAPI.onGet('/v1/me').replyOnce(401);
    mockAPI.onPost('/v1/auth/refresh').replyOnce(200, { session: newSession });
    mockAPI.onGet('/v1/me').replyOnce(200, { user: userData });

    const result = await apiClient.get('/v1/me');

    expect(result.data.user).toEqual(userData);
    expect(store.session.accessToken).toBe(newSession.accessToken);
  });
});

// test_onboarding_draft_persistence
describe('Onboarding Store', () => {
  it('should persist and restore draft', async () => {
    const store = useOnboardingStore.getState();

    store.updateProfile({ fullName: 'John Doe' });
    await new Promise(resolve => setTimeout(resolve, 100)); // Wait for persist

    // Reset store
    store.clearDraft();

    // Load persisted
    await store.loadPersistedDraft();

    expect(store.draft.profile.fullName).toBe('John Doe');
  });
});

```

7. Technology Stack & Libraries

7.1 Backend Dependencies

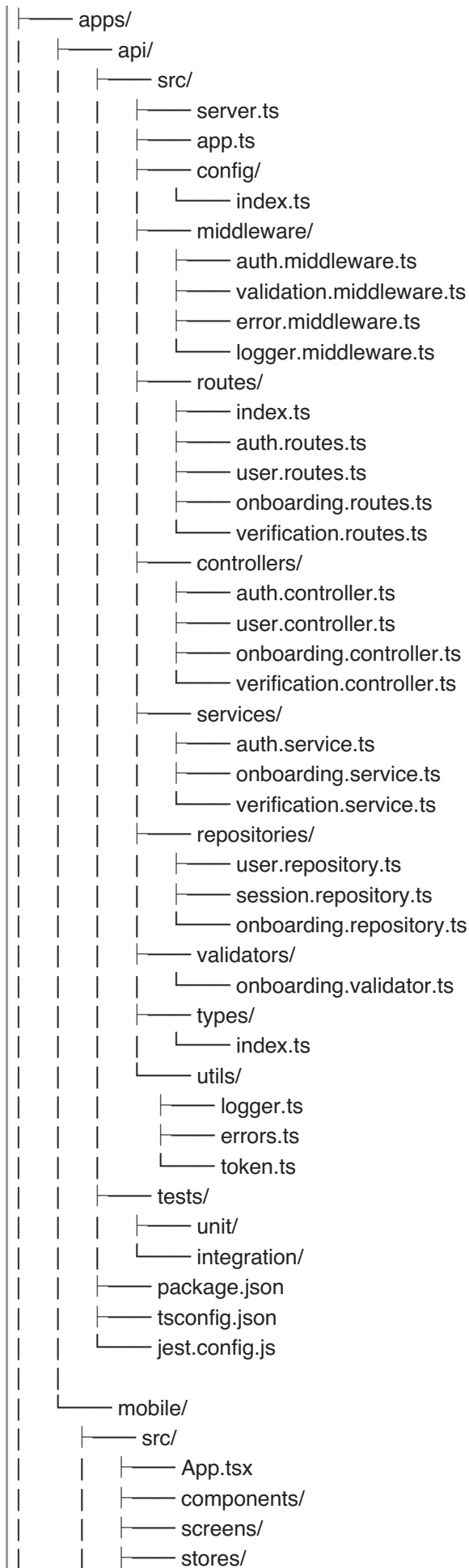

```
{
  "dependencies": {
    "express": "^4.18.2",
    "cors": "^2.8.5",
    "helmet": "^7.1.0",
    "express-validator": "^7.0.1",
    "jsonwebtoken": "^9.0.2",
    "bcrypt": "^5.1.1",
    "uuid": "^9.0.1",
    "winston": "^3.11.0"
  },
  "devDependencies": {
    "@types/express": "^4.17.21",
    "@types/node": "^20.10.0",
    "typescript": "^5.3.3",
    "ts-node": "^10.9.2",
    "nodemon": "^3.0.2",
    "jest": "^29.7.0",
    "supertest": "^6.3.3",
    "@types/jest": "^29.5.11"
  }
}
```

7.2 Mobile Dependencies

```
{
  "dependencies": {
    "expo": "~50.0.0",
    "react": "18.2.0",
    "react-native": "0.73.0",
    "@react-navigation/native": "^6.1.9",
    "@react-navigation/bottom-tabs": "^6.5.11",
    "@react-navigation/stack": "^6.3.20",
    "zustand": "^4.4.7",
    "axios": "^1.6.2",
    "@react-native-async-storage/async-storage": "^1.21.0",
    "expo-secure-store": "~12.8.1",
    "react-hook-form": "^7.48.2",
    "zod": "^3.22.4"
  },
  "devDependencies": {
    "@types/react": "~18.2.45",
    "typescript": "^5.3.3",
    "@testing-library/react-native": "^12.4.2",
    "jest": "^29.7.0"
  }
}
```

8. Project Structure (Monorepo)

mal.ai/





9. Implementation Phases

Phase 1: Project Setup (Milestone 1 Start)

- Initialize monorepo structure
- Set up TypeScript configs
- Install dependencies
- Create basic Express server
- Create basic Expo app
- Set up routing and navigation

Phase 2: Authentication (Milestone 1)

- Implement backend auth endpoints
- Add JWT token generation
- Create auth store in mobile
- Build login screen
- Implement secure storage
- Add route guards

Phase 3: Onboarding Flow (Milestone 1)

- Create 5-step onboarding screens
- Implement onboarding store
- Add form validation
- Implement draft persistence
- Build submit endpoint
- Connect submission flow

Phase 4: Verification & Theme (Milestone 1)

- Add verification status endpoint
- Create home screen
- Implement theme store
- Build settings screen
- Add theme toggle functionality

Phase 5: Production Hardening (Milestone 2)

- Add comprehensive validation
- Implement refresh-then-retry
- Enhance error handling
- Add structured logging
- Write critical tests
- Add request correlation IDs

Phase 6: Advanced Features (Milestone 3 - Optional)

- Choose one: Idempotency OR Async polling OR Offline queue OR OpenAPI

10. Key Design Decisions

10.1 State Management: Zustand

Why: Simpler than Redux, excellent TypeScript support, minimal boilerplate, easy persistence

10.2 Form Management: React Hook Form + Zod

Why: Type-safe validation, good performance, declarative schema validation

10.3 API Client: Axios

Why: Interceptor support for auth refresh, better error handling than fetch

10.4 Storage: In-Memory (Backend)

Why: Assessment requirement, fast, no DB setup needed

10.5 Logging: Winston

Why: Structured logging, multiple transports, production-ready

10.6 Token Strategy: JWT with Refresh

Why: Stateless auth, standard pattern, supports expiry and refresh flow

11. Performance Considerations

- Debounce form inputs to reduce store updates
- Memoize theme tokens to avoid recalculations
- Use React.memo for expensive components
- Implement request caching for verification status
- Add loading states to prevent duplicate requests
- Consider pagination if lists grow (not required for assessment)

12. Accessibility Considerations

- Proper label associations for inputs
- Keyboard navigation support

- Screen reader support (accessibilityLabel)
- Sufficient color contrast in themes
- Focus management in multi-step forms
- Error announcements

13. Success Criteria Summary

Milestone 1 (Must Have):

- Login flow works end-to-end
- Onboarding 5 steps complete
- Submit and status endpoints work
- Theme toggle functional
- Basic error handling
- No crashes in happy path

Milestone 2 (Should Have):

- Token refresh-then-retry works
- Route guards implemented
- Field-level validation errors
- Structured logging with correlation IDs
- Critical tests written
- Consistent error format

Milestone 3 (Nice to Have):

- Pick ONE advanced feature if time permits

This architecture provides a solid foundation for building a production-ready eKYC system that demonstrates fullstack competency, proper state management, security awareness, and testing discipline.